

AI & MACHINE LEARNING

Training Program — Lateral Induction Guide

DAY 1 & DAY 2

Foundations | Machine Learning | Deep Learning

Program AI/ML Lateral Induction	Audience Lateral hires — experienced professionals new to AI/ML	Duration Day 1 & Day 2 (Full Day each)
---	---	--

How to Use This Guide

This document is designed specifically for lateral hires joining an AI/ML team. The goal is not to make you a mathematician or a researcher — it is to give you the conceptual clarity and practical vocabulary so that you can work confidently alongside data scientists, ML engineers, and AI architects from Day 1.

For each topic, this guide covers:

- What it is — a plain-language explanation
- Why it matters — the business and technical reason it exists
- How it works — a conceptual walkthrough (without heavy math)
- Where it fits — how it connects to the bigger picture
- Common pitfalls — things that trip up beginners

A Note on Depth

This guide intentionally avoids going deep into algorithm internals, mathematical proofs, or hyper-technical details.

The priority is clarity, context, and confidence — understanding the WHY before the HOW.

Practical labs (Jupyter Notebooks) will accompany each day to give you hands-on experience.

DAY 1 — Foundations of AI, ML & Python Environment

Day 1 builds the foundation. Before writing a single line of code, it is critical to understand the landscape — what AI and ML actually are, where they are used, and why Python became the default language of this field. The second half of Day 1 focuses on getting your hands dirty with real data.

1.1 What Is Artificial Intelligence?

Artificial Intelligence (AI) is the broad field of building systems that can perform tasks that typically require human intelligence — things like understanding language, recognising images, making decisions, or playing games.

Think of AI as the umbrella. Under this umbrella live several sub-fields, including Machine Learning, Deep Learning, Natural Language Processing, Computer Vision, and Robotics.

Term	What It Means
Artificial Intelligence	The overall goal: machines that mimic human intelligence
Machine Learning	A subset of AI — machines that learn from data rather than explicit programming
Deep Learning	A subset of ML — uses brain-inspired networks (many layers) for complex tasks
NLP	Natural Language Processing — AI that reads and generates human language
Computer Vision	AI that processes and understands images and video

Why This Matters

Many people conflate AI with robots or science fiction. In practice, most AI you will encounter at work is narrower and more specific — a model that predicts customer churn, flags suspicious transactions, or recommends a product. Understanding what AI is (and is not) prevents inflated expectations and helps you have grounded conversations with stakeholders.

Real-World AI in Action

Gmail's Smart Reply — NLP predicts short reply suggestions as you read email

Netflix recommendations — ML models predict what you will enjoy watching next

Bank fraud detection — ML flags unusual transactions in real time

Google Maps ETAs — ML predicts travel time using historical traffic patterns
ChatGPT / Gemini — Large Language Models trained on massive text corpora

1.2 What Is Machine Learning — and Why Should You Care?

Traditional programming is rule-based. You write explicit instructions: 'if age > 60 and smoker is true, flag as high risk.' This breaks down fast — real-world patterns are too complex for humans to enumerate every rule.

Machine Learning flips the approach. Instead of writing rules, you feed the system examples (data) and let it discover the rules on its own. The system 'learns' which patterns in the data are most predictive of the outcome you care about.

Traditional Programming	Machine Learning
Rules + Data → Output	Data + Output → Rules (learned)
Humans write logic	Machine discovers logic
Brittle — breaks on unseen cases	Generalises to new data
Works well for known, fixed logic	Works well for patterns in large data
Tax calculator, form validator	Fraud detector, recommendation engine

1.3 Types of Machine Learning

There are three major types of machine learning. Each type matches a different kind of problem and data availability.

Supervised Learning

The most common type in practice. You have labelled training data — meaning every example in your dataset has both an input (features) and a known correct answer (label).

- Analogy: A teacher shows you 1,000 solved maths problems. You learn the pattern and solve new problems yourself.
- Examples: email spam detection (label: spam/not spam), house price prediction (label: price), loan default prediction (label: default/no default)
- Key requirement: You need historical data where the outcome is already known

Unsupervised Learning

You have data but no labels. The machine looks for hidden structure, groups, or patterns entirely on its own — without being told what to look for.

- Analogy: A librarian sorts thousands of unsorted books into groups by noticing similarities, without any instructions.
- Examples: customer segmentation (grouping customers by behaviour), anomaly detection (finding transactions that look unusual), topic modelling (discovering themes across documents)
- Key requirement: No labels needed — the model discovers structure autonomously

Reinforcement Learning

An agent learns by taking actions in an environment and receiving rewards or penalties. There is no dataset — the model learns by trial and error over many iterations.

- Analogy: Training a dog. Good behaviour gets a treat; bad behaviour gets a correction. Repeat thousands of times.
- Examples: game-playing AI (AlphaGo, chess engines), robot locomotion, autonomous driving, recommendation system optimisation
- Key requirement: A defined reward function and an environment to interact with — not used in standard Day 1 and Day 2 content

Which Type Is Most Common at Work?

For most enterprise ML use cases — customer analytics, risk scoring, demand forecasting, content moderation — Supervised Learning is the workhorse.

Unsupervised methods are widely used for exploratory analytics, customer segmentation, and anomaly detection.

Reinforcement Learning is specialised — you will encounter it primarily in robotics, gaming, and advanced recommendation systems.

1.4 The AI/ML Workflow — From Data to Decision

Every ML project — no matter how simple or complex — follows a recognisable sequence of steps. Understanding this workflow is critical because it tells you where problems come from and where to focus your effort.

Stage	What Happens Here
1. Problem Definition	What question are we answering? What does success look like? This is the most underrated step — a poorly defined problem wastes months of work.
2. Data Collection	Gather raw data from databases, APIs, logs, spreadsheets, etc. The quality and quantity of data here determines the ceiling of model quality.
3. Data Preprocessing	Clean, transform, and prepare the data. Handle missing values, encode categories, normalise numbers. In practice, this takes 60-80% of total project time.
4. Model Training	Feed processed data into an algorithm. The algorithm adjusts its internal parameters to minimise prediction error on the training data.
5. Model Evaluation	Test the trained model on data it has never seen. Measure accuracy, error rates, precision, recall, etc.
6. Tuning & Improvement	Adjust model configuration, add more data, or try different algorithms based on evaluation results.
7. Deployment	Put the model into a real application — as an API, a batch process, or an embedded system.
8. Monitoring	Track model performance in production. Data and user behaviour changes over time — models need ongoing maintenance.

Why Preprocessing Takes So Long

Real-world data is messy. Surveys have blank answers. Sales systems have duplicate records. Sensors produce corrupted readings. Dates are formatted differently across systems. Before a model can learn anything useful, the data must be made consistent and clean.

There is a saying in the field: 'Garbage in, garbage out.' A sophisticated algorithm on dirty data will always perform worse than a simple algorithm on clean data.

1.5 Why Python for AI/ML?

Python did not become the dominant AI/ML language by accident. Several factors combined to make it the obvious choice:

Reason	Explanation
Readable Syntax	Python reads almost like English. This lowers the barrier for scientists and analysts who are not professional software engineers.
Rich Ecosystem	Libraries like NumPy, Pandas, Scikit-learn, TensorFlow, PyTorch, and Hugging Face have been built specifically for data work. Nothing else comes close.
Interactive Notebooks	Jupyter Notebooks allow you to write code, run it, see results, and document explanations all in one place — ideal for exploration.
Community & Resources	Enormous community means Stack Overflow answers, tutorials, papers-with-code implementations, and library support in abundance.
Interoperability	Python connects easily to databases, web APIs, C/C++ extensions for performance, cloud services, and DevOps tooling.
Free & Open Source	No licensing costs. The entire ML toolchain is accessible to anyone.

Your Python Environment Options

For this training, we recommend Google Colab because it requires zero local installation and provides free GPU access for deep learning labs. However, it is worth knowing all available options:

Environment	Best For	Notes
Google Colab	Training labs, GPU access, sharing	Free, browser-based, connects to Google Drive. Recommended for this course.
Jupyter Notebook	Local exploration, presentations	Industry standard. Runs in browser, code stored in .ipynb files.
JupyterLab	More advanced local workflow	Enhanced Jupyter — file browser, multiple tabs, extensions.
Anaconda	Full local environment management	Bundles Python, Jupyter, and 250+ data science packages in one installer.
VS Code + Python	Production code development	Better for large codebases, version control, and debugging.
Spyder	MATLAB-like IDE experience	Variable explorer makes it familiar for engineers from MATLAB backgrounds.

1.6 Python Essentials for AI/ML

You do not need to be a Python expert on Day 1. You need to be fluent in the constructs you will encounter every single day when working with data. Here is a conceptual walkthrough of each.

Variables and Data Types

Python variables are dynamically typed — you do not declare the type; Python infers it. The types you will use most in ML are:

- `int` / `float` — numbers (age, price, temperature readings)
- `str` — text (customer names, product descriptions, error messages)
- `bool` — `True`/`False` (`is_fraud`, `is_active`, `converged`)
- `list` — ordered collection, allows duplicates: `[1, 2, 3, 'a']`
- `dict` — key-value store: `{'age': 30, 'city': 'Bengaluru'}`
- `None` — represents the absence of a value (like `NULL` in SQL)

Control Flow — Conditionals and Loops

These are how you add logic to your code. In ML, you use loops constantly — to iterate over rows, process batches of data, train for multiple epochs (passes over training data), or evaluate multiple models.

- **if / elif / else:** Branch execution based on a condition. Used for logic gating, threshold checks, rule application.
- **for loops:** Iterate over a list, range, or dataset. You will use these to process every item in a collection.
- **while loops:** Repeat until a condition is false. Training loops in neural networks often run until the error stops improving.
- **List comprehensions:** Compact, Pythonic way to build a new list by transforming or filtering an existing one. Very common in data work.

Functions

Functions let you wrap reusable logic under a name. In ML projects, functions are used to build preprocessing pipelines, define evaluation steps, wrap model training, and create utilities you call repeatedly throughout a notebook.

- **def keyword:** Defines a function. Parameters can have default values.
- **return:** Sends a result back to the caller. Without it, the function returns `None`.
- **Lambda functions:** Anonymous, one-line functions. Commonly used inside Pandas operations like `.apply()`.

Working with Files and I/O

ML projects always start with reading data from somewhere — CSV files, Excel files, JSON responses from APIs, or direct database queries. Python makes this straightforward.

- **Pandas (`pd.read_csv`, `pd.read_excel`):** The primary tool for reading tabular data into a DataFrame — a table-like structure you can slice, filter, and transform.
- **`open()` / `with` statement:** For reading raw text or JSON files. The 'with' block automatically closes the file handle.
- **`os` and `pathlib`:** For navigating the filesystem, listing directory contents, and building file paths reliably.

1.7 Data Preprocessing — The Most Important Skill in Practical ML

Model quality is capped by data quality. A brilliant algorithm on messy data produces poor results. A basic algorithm on clean, well-structured data often outperforms it. This is why data preprocessing is the most time-intensive and arguably most important skill in applied ML.

Handling Missing Data

Real datasets almost always have missing values. Rows where a field was not filled in, sensors that went offline, surveys with skipped questions. You must decide how to handle them — and your choice matters.

Strategy	When to Use It
Drop rows	When only a small percentage (<5%) of rows are affected and the missingness is random. Dangerous if missingness is not random — you introduce bias.
Mean / Median Imputation	Fill missing numbers with the column average (mean) or midpoint (median). Median is more robust when data has outliers.
Mode Imputation	Fill missing categorical values with the most frequent category. Simple but can create artificial imbalance.
Forward / Backward Fill	Used with time-series data — fill a missing value with the previous (or next) known value.
Model-based Imputation	Use another model to predict what the missing value should be, based on other features. Most accurate but most complex.

Encoding Categorical Data

Machine learning algorithms are mathematical — they work with numbers. But many features in real data are text categories: city names, product types, colour, grade. You must convert these to numbers before feeding them to a model.

- **Label Encoding:** Assigns an integer to each unique category. 'Low'=0, 'Medium'=1, 'High'=2. Suitable when the categories have a natural order (ordinal data). Dangerous for unordered categories because it implies a relationship (e.g., Red=0, Blue=1, Green=2 implies Red < Blue, which is nonsensical).
- **One-Hot Encoding:** Creates a new binary column for every unique category. City='Mumbai' becomes [1,0,0]; City='Delhi' becomes [0,1,0]. No false ordering is implied. Downside: creates many columns if the category has hundreds of unique values.

Feature Scaling

Many algorithms — particularly those that use distance calculations (KNN), gradient descent (Linear Regression, Neural Networks), or regularisation (Ridge/Lasso) — are sensitive to the scale of input features.

If one feature is 'age' (0-100) and another is 'salary' (20,000-500,000), the salary feature will dominate any distance calculation purely because its numbers are larger — not because it is more important. Scaling corrects this.

Scaling Method	How It Works and When to Use
StandardScaler (Z-score)	Transforms each value to have mean=0 and standard deviation=1. Best when data is roughly normally distributed. Most common choice for regression and neural networks.
MinMaxScaler	Compresses all values to a [0,1] range. Best when you know the bounds and the distribution is not Gaussian. Common for neural networks and image pixel values.
RobustScaler	Uses the median and interquartile range instead of mean and std — less affected by outliers. Use when your data has significant outliers.

Train / Test Split

Before training any model, you split your dataset into two parts:

- **Training set (typically 70-80% of data):** The model learns from this data. It sees both the features and the labels.
- **Test set (typically 20-30% of data):** Held back and never used during training. Used only to measure how well the model generalises to new, unseen data.

The reason for this split is fundamental: if you test on the same data you trained on, you will measure memorisation, not learning. A student who memorises the exam questions will score 100% — but may fail entirely on different questions.

Critical Insight: Data Leakage

Data leakage is one of the most common and damaging mistakes in ML projects.

It happens when information from the test set inadvertently influences model training — inflating evaluation scores and leading to poor real-world performance.

Always apply preprocessing (scaling, encoding) after the train/test split. If you scale before splitting, the test data has influenced the scaler — that is leakage.

In production, always fit transformers on training data only, then apply (transform) to test and new data.

1.8 Day 1 Session Schedule & Notebook Mapping

Time Block	Topics	Notebook
Morning (First Half)	AI/ML Overview, Types of Learning, Real-World Use Cases, The ML Workflow	Notebook 1A: Concepts & Exploration
Morning (First Half)	Why Python, Python Environment Setup, Python Essentials — Variables, Loops, Functions, I/O	Notebook 1A: Concepts & Exploration
Afternoon (Second Half)	Data Preprocessing: Missing Values, Encoding, Scaling	Notebook 1B: Data Preprocessing
Afternoon (Second Half)	Train/Test Split, Handling a Real CSV Dataset End to End	Notebook 1B: Data Preprocessing

Day 1 Outcomes — What You Should Be Able to Do

Explain the difference between AI, ML, and Deep Learning without confusion
Describe the three types of learning (supervised, unsupervised, reinforcement) with real examples
Walk through the ML workflow from problem definition to deployment
Write and run Python code in Google Colab — variables, loops, functions, file I/O
Load a CSV dataset into a Pandas DataFrame and inspect it
Handle missing data using drop, mean/median/mode imputation
Encode categorical columns using LabelEncoder and OneHotEncoder
Scale numerical features using StandardScaler and MinMaxScaler
Split a dataset into training and test sets correctly (after preprocessing)

DAY 2 — Machine Learning Algorithms & Introduction to Deep Learning

Day 2 moves from concepts to models. You will build your first machine learning models, learn how to measure how well they perform, understand why models sometimes fail, and get your first exposure to neural networks. The focus is on understanding what each model is doing and why — not on mastering every algorithm.

2.1 Regression — Predicting Continuous Values

Regression is the simplest and most interpretable family of machine learning models. The task is to predict a continuous numerical output — not a category, but a number.

- How many units will we sell next month?
- What is this house worth?
- How many days until a machine component fails?
- What will a customer's lifetime value be?

Simple Linear Regression

The most fundamental model in statistics and ML. Given one input feature (X), it fits a straight line to predict one output (Y). The model finds the slope and intercept that minimise the total prediction error across all training examples.

- **Conceptual model:** $Y = (\text{slope} \times X) + \text{intercept}$
- **Training means:** Finding the slope and intercept that minimise prediction error
- **Good for:** Understanding directional relationships, quick baselines, interpretability
- **Limitation:** Assumes the relationship is linear — if it is curved or complex, the model will underfit

Multiple Linear Regression

The real-world extension: use multiple input features to predict one output. A house price is not determined by size alone — it depends on location, age, number of bedrooms, nearby schools, and dozens of other factors.

- **Conceptual model:** $Y = (w_1 \times X_1) + (w_2 \times X_2) + \dots + (w_n \times X_n) + \text{intercept}$
- **Training means:** Finding the best weights (coefficients) for each feature
- **Good for:** Multi-factor prediction problems, highly interpretable — you can read off the importance of each feature directly
- **Limitation:** Assumes linear relationships between all features and the target; sensitive to multicollinearity (when input features are correlated with each other)

How Do We Know the Model Is Good? — Regression Metrics

You need numbers to measure how accurate your model is. For regression, the most common metrics are:

Metric	What It Measures and Why It Matters
Mean Absolute Error (MAE)	Average absolute difference between prediction and actual. Easy to interpret — if MAE = 5,000 for house prices, your predictions are off by \$5,000 on average.
Mean Squared Error (MSE)	Average squared difference. Penalises large errors more than small ones. Used during training because it is mathematically smooth to optimise.
Root Mean Squared Error (RMSE)	Square root of MSE. Returns the error in original units — interpretable like MAE but penalises large errors more.
R-squared (R^2)	Proportion of variance in the target explained by the model. Ranges from 0 to 1 (higher is better). $R^2 = 0.85$ means the model explains 85% of the variation in outcomes.
Adjusted R^2	Penalises adding features that do not improve the model. Preferable to R^2 when comparing models with different numbers of features.

2.2 Classification — Predicting Categories

Classification is supervised learning where the output is a category — a discrete label — rather than a number. This covers an enormous range of business problems:

- Will this customer churn? (Yes / No)
- Is this email spam? (Spam / Not Spam)
- Which product category does this item belong to? (A / B / C / D)
- Is this medical image showing signs of disease? (Positive / Negative)

Day 2 introduces three classification algorithms — not to master each, but to understand their character: when they shine, and when they struggle.

Logistic Regression

Despite the misleading name, Logistic Regression is a classification algorithm. It predicts the probability that an input belongs to a class (0 or 1). A threshold (usually 0.5) converts that probability into a categorical prediction.

- **Analogy:** A doctor assigns a 0-100% probability of disease based on test results, then says 'above 60%, we treat it as positive.'
- **Strengths:** Fast to train, highly interpretable, gives probabilities not just binary outputs, good baseline for any binary classification problem
- **Limitations:** Assumes a roughly linear decision boundary — struggles when classes are separated by complex curved boundaries
- **Common use:** Credit risk scoring, medical diagnosis, email spam detection, churn prediction

K-Nearest Neighbours (KNN)

KNN is one of the most intuitive algorithms in ML. It makes no assumptions about the data's underlying distribution. To classify a new data point, it simply finds the K most similar points in the training set and assigns the majority class.

- **Analogy:** Asking your K closest neighbours for a restaurant recommendation. If 4 out of 5 neighbours recommend Thai food, you go for Thai.
- **Strengths:** No training phase (lazy learner), naturally handles non-linear boundaries, easy to understand and implement
- **Limitations:** Very slow at prediction time on large datasets (it must compare to every training point), sensitive to irrelevant features, sensitive to scale (must standardise features before using KNN)
- **Key parameter:** K — the number of neighbours. Small K = complex boundary, prone to overfitting. Large K = smooth boundary, prone to underfitting.

Decision Tree

A decision tree builds a flowchart-like structure — a series of if/then questions — that splits the data step by step to separate the classes. At each step, it chooses the question (feature + threshold) that best separates the data.

- **Analogy:** The 20 Questions game. Each question progressively narrows down the possibilities until you arrive at the answer.
- **Strengths:** Extremely interpretable — you can literally print the tree and show it to a non-technical stakeholder. Handles both numerical and categorical features. No scaling needed.
- **Limitations:** Individual trees tend to overfit — they grow too deep and memorise the training data. This is why decision trees are often used as building blocks inside more powerful ensemble methods (Random Forest, Gradient Boosting).
- **Common use:** Customer segmentation, medical decision support, fraud rule extraction

Classification Metrics — Beyond Accuracy

Accuracy (percentage correct) is intuitive but deceptive. In a dataset where 95% of transactions are legitimate and 5% are fraud, a model that predicts 'not fraud' for everything achieves 95% accuracy — while being completely useless.

You need a richer set of metrics:

Metric	What It Measures
Accuracy	Overall percentage of correct predictions. Misleading on imbalanced datasets.
Precision	Of all the cases the model predicted as Positive, what fraction was actually Positive? High precision = few false alarms.
Recall (Sensitivity)	Of all actual Positive cases, what fraction did the model catch? High recall = few missed cases. Critical in medical diagnosis, fraud detection.
F1 Score	Harmonic mean of Precision and Recall. Good single metric when you need balance between both. Values closer to 1 are better.
Confusion Matrix	A 2x2 (or NxN) table showing True Positives, False Positives, True Negatives, False Negatives. The foundation from which all other metrics are calculated.
ROC-AUC	Area Under the ROC Curve. Measures how well the model distinguishes between classes across all possible thresholds. AUC = 0.5 is random; AUC = 1.0 is perfect.

Precision vs. Recall — The Fundamental Trade-Off

FRAUD DETECTION: Missing a fraud case (low recall) is very costly. You need high recall — catch everything suspicious, even if that means some false alarms.

EMAIL SPAM FILTER: Wrongly marking an important email as spam (low precision) is very costly. You need high precision — only flag something as spam if you are very sure.

MEDICAL SCREENING: Missing a cancer diagnosis (low recall) is catastrophic. High recall is paramount — better to over-refer than to miss a case.

In practice, increasing one often decreases the other. The choice of threshold and metric depends entirely on the cost of each type of error.

2.3 Overfitting, Underfitting & Cross-Validation

These three concepts are central to everything in ML. Understanding them at an intuitive level is more important than memorising the mathematics.

The Core Problem: Generalisation

The goal of a machine learning model is not to perform perfectly on training data. It is to perform well on new, unseen data. The ability to do so is called generalisation. Two failure modes stand in the way:

Overfitting	Underfitting
Model too complex for the data	Model too simple for the data
Memorises training data — including noise	Misses the real pattern entirely
Performs great on training, poorly on test	Performs poorly on both training and test
Analogy: Student memorises every past exam without understanding	Analogy: Student barely studied and guesses randomly
Fix: Add more data, reduce model complexity, regularisation, Dropout	Fix: Use a more complex model, add more relevant features

Cross-Validation — More Reliable Than a Single Train/Test Split

A single train/test split can be misleading. If you happen to get an easy test set by chance, your accuracy metric will look better than reality. Cross-validation is a more rigorous technique.

K-Fold Cross-Validation (most common): Split the entire dataset into K equal parts (folds). Train on K-1 folds, test on the remaining fold. Repeat K times, rotating which fold is held out each time. Report the average performance across all K runs.

- Why it is better: Every data point is used for testing exactly once. The reported metric is an average across K experiments, making it far less sensitive to lucky or unlucky splits.
- Common K values: K=5 (5-fold) or K=10 (10-fold) are standard choices.
- Cost: Training K models instead of 1 — but worth it for reliable estimates, especially with smaller datasets.

2.4 Introduction to Artificial Neural Networks (ANN)

Neural networks are the backbone of modern deep learning. Before understanding transformers, LLMs, or image recognition systems, you need to understand how a basic neural network works. This section provides a conceptual introduction — no heavy maths.

Biological Inspiration

The brain processes information through billions of neurons connected by synapses. When a neuron receives enough signal from its neighbours, it fires and passes the signal onward. Artificial neural networks are a very loose mathematical analogy of this structure.

Each artificial neuron takes in numerical inputs, multiplies each by a weight (importance), adds them together, applies an activation function, and passes the result forward. Billions of these neurons, arranged in layers, can learn extraordinarily complex patterns.

The Architecture of a Neural Network

A standard (feedforward) neural network consists of:

Layer	Role
Input Layer	Receives the raw features. One neuron per input feature. No computation here — just data entry.
Hidden Layer(s)	Where learning happens. Each hidden neuron applies weights + activation to transform the representation. More layers = more complex patterns can be learned.
Output Layer	Produces the prediction. One neuron for regression. One neuron per class for multi-class classification (with softmax activation producing probabilities).

Activation Functions — Why Non-Linearity Matters

Without activation functions, stacking multiple layers of linear operations would still produce a linear model — no matter how deep. Activation functions introduce non-linearity, allowing the network to learn curved, complex decision boundaries.

Activation Function	What It Does and Where It Is Used
ReLU (Rectified Linear Unit)	Outputs zero for negative inputs; passes positive inputs unchanged. The default hidden-layer activation for most modern networks. Simple, fast, works very well in practice.
Sigmoid	Squashes output to (0, 1). Historically used in hidden layers; today mainly used in the output layer for binary classification to produce a probability.

Activation Function	What It Does and Where It Is Used
Tanh	Squashes output to (-1, 1). Stronger gradients than sigmoid; occasionally used in RNNs. Generally replaced by ReLU in deep networks.
Softmax	Converts a vector of raw scores to probabilities that sum to 1. Used in the output layer for multi-class classification.

How a Neural Network Learns — Training in Plain English

Training a neural network is an iterative process of making predictions, measuring error, and adjusting weights to reduce that error:

1. **Forward Pass:** Input features flow forward through the network. Each layer applies weights and activation. The output layer produces a prediction.
2. **Loss Calculation:** Compare the prediction to the true label using a loss function (e.g., Mean Squared Error for regression, Cross-Entropy for classification). This produces a single number — the error.
3. **Backward Pass (Backpropagation):** Calculate how much each weight contributed to the error, using the chain rule of calculus. This is handled automatically by frameworks like Keras and PyTorch.
4. **Weight Update (Gradient Descent):** Adjust each weight slightly in the direction that reduces the loss. The learning rate controls how large each adjustment is.
5. **Repeat:** One complete pass through the entire training set is called an epoch. Training runs for many epochs — typically 10 to 500 — until the loss stops improving.

The Learning Rate — A Critical Hyperparameter

Too HIGH: Weights overshoot the optimal values — the model oscillates and may diverge (loss goes up instead of down).

Too LOW: Training is extremely slow — the model makes tiny steps and may take thousands of epochs to converge.

Just right: The model converges steadily within a reasonable number of epochs.

In practice, values between 0.0001 and 0.01 are common starting points. Adaptive optimisers like Adam adjust the learning rate automatically per parameter.

Key Hyperparameters in Neural Networks

Unlike the weights (which the model learns), hyperparameters are design choices you make before training:

Hyperparameter	What It Controls
----------------	------------------

Hyperparameter	What It Controls
Number of Layers	Depth of the network. More layers can model more complex patterns but require more data and more training time.
Neurons per Layer	Width of the network. More neurons = more capacity to learn, but also more risk of overfitting.
Learning Rate	Step size during weight updates. Critical — too high or too low causes training to fail or be very slow.
Batch Size	Number of training examples processed in each forward+backward pass before a weight update. Larger batches are more stable but require more memory.
Epochs	How many complete passes over the training data. Too few = underfitting. Too many = overfitting (use early stopping to counter this).
Dropout Rate	Randomly disables a fraction of neurons during each training step. Acts as regularisation — forces the network to build redundant representations and prevents overfitting.

Dropout — Preventing Overfitting in Neural Networks

Dropout is one of the most effective regularisation techniques for deep neural networks. During training, at each forward pass, a random subset of neurons is temporarily turned off. This prevents any single neuron from becoming too important and forces the network to develop more robust, distributed representations.

- **Typical dropout rates:** 0.2 to 0.5 (20% to 50% of neurons dropped per step)
- **Important:** Dropout is only active during training. During inference (making real predictions), all neurons are active and their outputs are scaled accordingly.

Keras and TensorFlow — Your Neural Network Toolkit

TensorFlow is Google's open-source library for numerical computation and ML. Keras is its high-level API — it allows you to build, compile, and train neural networks with a small number of clean, readable lines of code.

- **model = Sequential():** Defines a feedforward network layer by layer
- **model.add(Dense(64, activation='relu')):** Adds a dense (fully connected) hidden layer with 64 neurons
- **model.compile(optimizer='adam', loss='mse'):** Sets the optimiser and loss function
- **model.fit(X_train, y_train, epochs=50):** Trains the model for 50 epochs
- **model.evaluate(X_test, y_test):** Reports loss and metrics on the test set

2.5 Day 2 Session Schedule & Notebook Mapping

Time Block	Topics	Notebook
Morning (First Half)	Regression — Linear & Multiple Linear Regression, Model Evaluation (MAE, MSE, R^2)	Notebook 2A: Regression & Classification
Morning (First Half)	Classification — Logistic Regression, KNN, Decision Tree; Metrics — Confusion Matrix, F1, ROC-AUC	Notebook 2A: Regression & Classification
Afternoon (Second Half)	Overfitting vs Underfitting, Cross-Validation; ANN Architecture and How Training Works	Notebook 2B: Neural Networks
Afternoon (Second Half)	Implementing an ANN in Keras, Dropout, Visualising Training Curves, Model Tuning	Notebook 2B: Neural Networks

Day 2 Outcomes — What You Should Be Able to Do

Build a linear regression model in Scikit-learn and evaluate it using MAE, MSE, and R^2
Build and evaluate a classification model using Logistic Regression, KNN, or Decision Tree
Interpret a confusion matrix and explain what precision, recall, and F1 score each measure
Explain what overfitting and underfitting are and how to detect them via learning curves
Implement K-Fold Cross-Validation and explain why it is more reliable than a single split
Describe a neural network's architecture — input, hidden, and output layers
Explain forward pass, loss calculation, backpropagation, and gradient descent in plain language
Build a simple ANN in Keras for a regression problem
Add Dropout to a network and explain what it does
Visualise training and validation loss curves and diagnose underfitting/overfitting from them

Appendix — Glossary of Key Terms

A quick-reference glossary for the most important terms introduced across Day 1 and Day 2.

Term	Plain-Language Definition
Algorithm	A set of instructions a model follows to learn from data. Examples: linear regression, decision tree, neural network.
Backpropagation	The process of calculating how much each weight in a neural network contributed to the prediction error, used to update weights.
Batch Size	The number of training examples the model processes before updating its weights. Larger batches are more stable; smaller batches introduce helpful noise.
Classification	A supervised learning task where the output is a category (class label), not a number.
Cross-Validation	A technique for reliably estimating model performance by training and testing on different subsets of the data multiple times.
Data Leakage	When information from the test set contaminates the training process, leading to inflated performance metrics.
Dropout	A regularisation technique that randomly deactivates neurons during training to prevent overfitting.
Epoch	One complete pass through the entire training dataset during neural network training.
Feature	An individual measurable input variable used by a model (e.g., age, salary, number of transactions).
Gradient Descent	The optimisation process that adjusts model weights in small steps to minimise the loss function.
Hyperparameter	A configuration setting chosen before training that controls how the model learns (e.g., learning rate, number of layers, K in KNN).
Label / Target	The known correct output in a supervised learning dataset (what the model is trying to predict).
Learning Rate	Controls how large each weight update step is during gradient descent training.
Loss Function	A mathematical formula that measures how wrong a model's predictions are. Training aims to minimise this.
Model	A mathematical function trained on data that makes predictions on new inputs.
Overfitting	When a model performs very well on training data but poorly on new data — it has memorised rather than learned.

Term	Plain-Language Definition
Preprocessing	All data cleaning and transformation steps before feeding data into a model.
Regression	A supervised learning task where the output is a continuous number (not a category).
Regularisation	Techniques that prevent overfitting by constraining model complexity (Dropout, L1/L2 penalty).
Scikit-learn	Python's primary library for traditional machine learning algorithms — regression, classification, clustering, preprocessing.
Supervised Learning	Learning from labelled data — the model learns to map inputs to known outputs.
Test Set	Data held back from training, used only to evaluate final model performance on unseen examples.
Training Set	The portion of data the model learns from during the training process.
Underfitting	When a model is too simple to capture the patterns in the data — performs poorly on both training and test data.
Weight	A learnable parameter in a model. During training, weights are adjusted to minimise prediction error.

— **End of Day 1 & Day 2 Reference Guide** —

Continue to Day 3: GenAI Foundations — Transformers, LLMs & Prompt Engineering